

Robot Dynamics – MATLAB

151-0851-00L • ETH Zürich, RSL (Prof. Hutter) • HS 2026

1 MATLAB Essentials

1.1 Workspace & exercise framework

```
init_workspace; % addpath(genpath(exerciseFolder))
loadvars; % creates object abbRobot
abbRobot.setJointPositions(q); % q: 6x1
evaluate_problems; % compares to *.solution.p
which jointToPosJac % which file is on path?
out = feval('jointToPosJac', q); % call by name (string)
clc; clear; close all; % fresh start
```

1.2 Vectors, matrices, indexing

```
q = zeros(6,1); v = [1;2;3]; % COLUMN vectors!
A = eye(3); B = blkdiag(A,2*A); Z = zeros(3,6);
T(1:3,1:3) % rotation part T(1:3,4) position
J(:,k) % k-th column J(end,:) last row
x(end+1) = 5; % append
A' % conj. transpose A.' % plain transpose
A*B % matrix product A.*B % element-wise
A\b % solve A*x = b (prefer over inv(A)*b)
b/A % solve x*A = b
numel(q), size(J,2), length(q), reshape(v,3,[])
v(:) % force column; repmat(v,1,4)
dot(a,b), cross(a,b), norm(v), trace(C), det(C)
kron(A,B), diag(v), triu(A), rank(J), cond(J)
[U,S,V] = svd(J); [V,D] = eig(M); N = null(J);
```

1.3 Functions

```
function [out1,out2] = myFun(q, varargin)
    if nargin < 2, k = 1; else, k = varargin{1}; end
    out1 = k*q; out2 = [];
end
f = @(x) x.^2 + 1; % anonymous function
Rz = @(a) [cos(a) -sin(a) 0; sin(a) cos(a) 0; 0 0 1];
% local functions go at the END of a function file
```

1.4 Control flow, structs, cells, printing

```
for k = 1:6, ... end; while err > tol, ... end
if norm(e) < 1e-6 && it < 100, ... elseif ..., else ... end
s.m = 2; s.I = eye(3); params = struct('m',2,'I',0.3);
c = {'jointToPosJac','jointToRotJac'}; fun = c{1};
fprintf('%s: err = %.3e\n', fun, err); disp(A)
tic; ...; t = toc; % timing
assert(norm(A-B) < 1e-9, 'mismatch');
isequal(size(A),[3 3]) % shape checks
```

Classic bugs: row vs. column vectors; ' on complex/sym (use ' '); * vs. .*; indexing starts at 1; == on floats (use tolerance); q(3) inside jointToTransform23 when q is a scalar (guard with if length(q)>1).

2 Rotations

2.1 Elementary rotations & skew matrix

```
Cx = @(a) [1 0 0; 0 cos(a) -sin(a); 0 sin(a) cos(a)];
Cy = @(a) [cos(a) 0 sin(a); 0 1 0; -sin(a) 0 cos(a)];
Cz = @(a) [cos(a) -sin(a) 0; sin(a) cos(a) 0; 0 0 1];
skew = @(v) [0 -v(3) v(2); v(3) 0 -v(1); -v(2) v(1) 0]; % skew(a)*b = cross(a,b)
unskew = @(S) [S(3,2); S(1,3); S(2,1)];
```

Properties: C⁻¹ = C^T, det C = 1, CAC = C_{AB}C_{BC}, CBA = C_{AB}^T.

isRot = norm(C'*C - eye(3)) < 1e-9 && abs(det(C)-1) < 1e-9;

2.2 Euler angles

ZYX (yaw-pitch-roll) C = C_z(z)C_y(y)C_x(x); XYZ C = C_x(x)C_y(y)C_z(z)

% ZYX -> chi = [x; y; z]

z = atan2(C(2,1), C(1,1));

y = atan2(-C(3,1), sqrt(C(3,2)^2 + C(3,3)^2));

x = atan2(C(3,2), C(3,3));

% XYZ -> chi = [x; y; z]

y = atan2(-C(2,3), C(3,3));

x = atan2(C(1,3), sqrt(C(1,1)^2 + C(1,2)^2));

z = atan2(-C(1,2), C(1,1));

wrap = @(a) atan2(sin(a), cos(a)); % to (-pi, pi]

Always atan2, never atan/asin alone. Singular at $y = \pm\pi/2$ (gimbal lock).

2.3 Angle-axis / rotation vector

$C = \cos\theta I + \sin\theta [n]_{\times} + (1 - \cos\theta) nn^T, \quad \varphi = \theta n$

function C = rotVecToRotMat(phi)

th = norm(phi);

if th < 1e-12, C = eye(3); return; end

n = phi/th; S = [0 -n(3) n(2); n(3) 0 -n(1); -n(2) n(1) 0];

C = cos(th)*eye(3) + sin(th)*S + (1-cos(th))*(n*n')

end

function phi = rotMatToRotVec(C)

th = acos(max(-1, min(1, (trace(C)-1)/2))); % clamp!

if th < 1e-12, phi = zeros(3,1); return; end

n = 1/(2*sin(th)) * [C(3,2)-C(2,3); C(1,3)-C(3,1); C(2,1)-C(1,2)];

phi = th*n;

end

Near $\theta = \pi$ the $1/\sin\theta$ formula is ill-conditioned – go via quaternion or eigenvector of $C - [V,D]=\text{eig}(C)$, eigenvalue 1).

2.4 Unit quaternions $\xi = [\xi_0;\xi] = [\cos\frac{\theta}{2}; n\sin\frac{\theta}{2}]$

function p = quatMult(q, r) % q (w) r

q0 = q(1); qv = q(2:4); r0 = r(1); rv = r(2:4);

p = [q0*r0 - qv'*rv;

q0*rv + r0*qv + cross(qv, rv)];

end

qConj = @(q) [q(1); -q(2:4)]; % = inverse (unit)

function C = quatToRotMat(q)

w = q(1); n = q(2:4);

C = (2*w^2-1)*eye(3) + 2*w*skew(n) + 2*(n*n');

end

function q = rotMatToQuat(C)

J = 0.5*[sqrt(1+trace(C));

sign(C(3,2)-C(2,3))*sqrt(C(1,1)-C(2,2)-C(3,3)+1);

sign(C(1,3)-C(3,1))*sqrt(C(2,2)-C(3,3)-C(1,1)+1);

sign(C(2,1)-C(1,2))*sqrt(C(3,3)-C(1,1)-C(2,2)+1)];

end

Ar = quatToRotMat(q_AB)*Br; % rotVecWithQuat

% equivalently: [0;Br] * q_AB (w) [0;Br] (w) conj(q_AB)

C = q/norm(q); % re-normalise!

Matrix form: $\xi_1 \otimes \xi_2 = M_L(\xi_1) \xi_2 = M_R(\xi_2) \xi_1$

Ml = @(q) [q(1) -q(2:4)'; q(2:4) q(1)*eye(3)+skew(q(2:4))];

Mr = @(q) [q(1) -q(2:4)'; q(2:4) q(1)*eye(3)-skew(q(2:4))];

ξ and $-\xi$ are the same rotation. sign(0)=0 in rotMatToQuat can break the 180° case – check with norm(q). Robust alternative: pick largest diagonal (Shepperd).

3 Homogeneous Transformations & FK

$T_{AB} = \begin{bmatrix} C_{AB} & A^rAB \\ 0 & 1 \end{bmatrix}, T_{AB}^{-1} = \begin{bmatrix} C_{AB}^T & -C_{AB}^T A^rAB \\ 0 & 1 \end{bmatrix}$

Tnk = @(C,r) [C, r(1); 0 0 0 1];

Tinv = @(T) [T(1:3,1:3)', -T(1:3,1:3)*T(1:3,4); 0 0 0 1];

Ar_h = T_AB * [Br; 1]; Ar = Ar_h(1:3); % point map

% direction vectors: use C only (no translation!)

3.1 Forward kinematics chain (ABB IRB120)

function T_IE = jointToTransformIE(q)

T_IE = getTransformI0() * jointToTransform01(q) * ...

jointToTransform12(q) * jointToTransform23(q) * ...

jointToTransform34(q) * jointToTransform45(q) * ...

jointToTransform56(q) * getTransform6E();

end

I_r_IE = T_IE(1:3,4); C_r_IE = T_IE(1:3,1:3);

quat = rotMatToQuat(C_r_IE); % jointToQuat

Each joint transform: rotation about joint axis in its local frame + constant offset, e.g.

T01 = [Cz(q(1)), [0;0;0.145]; 0 0 0 1]; % z-axis joint

T12 = [Cy(q(2)), [0;0;0.145]; 0 0 0 1]; % y-axis joint

T34 = [Cx(q(4)), [0.134;0;0.070]; 0 0 0 1];

3.2 Store all intermediate frames (for Jacobians)

function [T_Ik] = allTransforms(q)

Tk = [getTransformI0(), jointToTransform01(q), ...

jointToTransform12(q), jointToTransform23(q), ...

jointToTransform34(q), jointToTransform45(q), ...

jointToTransform56(q), getTransform6E()];

end

jointToTransform34(q), jointToTransform45(q), ...

jointToTransform56(q)];

T_Ik = cell(numel(Tk),1); T = eye(4);

for k = 1:numel(Tk)

T = T*T{k,k}; T_Ik{k} = T; % T_I0, T_I1 ... T_I6

end

end

4 Differential Kinematics

$w_E = \begin{bmatrix} I^VE \\ I^W E \end{bmatrix} = J_0(q)\dot{q}, J_0 = \begin{bmatrix} J^P \\ J^R \end{bmatrix}$ (geometric), $\dot{X}_E = J_{e,A} \dot{q}$ (analytical)

4.1 Geometric Jacobian (revolute joints)

$I^P P_i = I^{n_i} \times (I^R I_E - I^R I_i); I^J R_i = I^{n_i}$

Prismatic: $I^J P_i = I^{n_i}, I^J R_i = 0$.

function [J_P, J_R] = geomJacobian(q)

T = allTransforms(q); % {T_I0,...,T_I6}

TE = T{end}*getTransform6E(); r_IE = TE(1:3,4);

n_loc = [0 0 1; 0 1 0; 0 1 0; 1 0 0; 0 1 0; 1 0 0]'; %

axes in frame k

J_P = zeros(3,6); J_R = zeros(3,6);

for k = 1:6

C_Ik = T{k+1}(1:3,1:3); r_Ik = T{k+1}(1:3,4);

n_k = C_Ik * n_loc(:,k); % joint axis in I

J_P(:,k) = cross(n_k, r_IE - r_Ik);

J_R(:,k) = n_k;

end

end

Joint k is between frames $k-1$ and k : its axis/origin is expressed via $T_{I,k}$ ($T\{k+1\}$ in the 1-based cell above, since $T(1)=T_{I0}$). Axis local to frame k = the axis used inside $\text{jointToTransform}(k-1)(k)$.

4.2 Numerical check (finite differences)

function J = numJac(f, q, h) % f: q -> R^m

if nargin < 3, h = 1e-7; end

f0 = f(q); J = zeros(numel(f0), numel(q));

for k = 1:numel(q)

dq = zeros(size(q)); dq(k) = h;

J(:,k) = (f(q+dq) - f(q-dq)) / (2*h); % central

end

end

JP_num = numJac(@(jointToPosition, q);

norm(JP_num - jointToPosJac(q)) % ~1e-9 expected

Rotational check: $[\omega]_{\times} = \dot{C} C^T$

dq = randn(6,1); h = 1e-7;

Cd = (jointToRotMat(q+h*dq) - jointToRotMat(q-h*dq))/(2*h);

W = Cd * jointToRotMat(q)'; w = unskew(W);

norm(w - jointToRotJac(q)*dq)

4.3 Analytical Jacobian & rate mappings

Write $J^{-1} = E\omega(X_R)\dot{X}_R \Rightarrow J_{A,R} = E\omega^{-1}J_R$ (check which of E_R/E_R^{-1} your script calls which!).

$ZYX, X_R = [z; y; x]; I\omega = \begin{bmatrix} 0 & -s_z & c_y c_z \\ 0 & c_z & c_y s_z \\ 1 & 0 & -s_y \end{bmatrix} \begin{bmatrix} \dot{z} \\ \dot{y} \\ \dot{x} \end{bmatrix}$

E_w = @(z,y) [0 -sin(z) cos(y)*cos(z);

0 cos(z) cos(y)*sin(z);

1 0 -sin(y)]; % omega = E_w*

chidot

chidot = E_w(z,y) \ omega; % singular at y+=pi/2

Quaternion rates: $\dot{\xi}_{IB} = \frac{1}{2} \begin{bmatrix} 0 \\ I\omega \end{bmatrix} \otimes \xi_{IB} = \frac{1}{2} \xi_{IB} \otimes \begin{bmatrix} 0 \\ B\omega \end{bmatrix}$

qdot = 0.5*quatMult([0; I_w], q_IB);

q_IB = q_IB + dt*qdot; q_IB = q_IB/norm(q_IB);

4.4 Symbolic Jacobian

syms q1 q2 q3 q4 q5 q6 real

q = [q1;q2;q3;q4;q5;q6]; % or: q = sym('q',[6 1],'real');

T = jointToTransformIE(q); % works if fns use only +,*,

sin,cos

r = T(1:3,4);

JP = simplify(jacobian(r, q));

JP_fun = matlabFunction(JP, 'Vars', {q}, 'File', 'JP_gen');

double(subs(JP, q, [0.1;0;0;0;0;0])) % numeric eval

Inside functions that should also take sym inputs: don't preallocate with zeros (numeric type) – use sym(zeros(3,6)) or build by concatenation.

5 Inverse (Differential) Kinematics

$J^+ = J^T(JJ^T)^{-1}$ (fat, full row rank); damped: $J^T(JJ^T + \lambda 2I)^{-1}$

Jp = pinv(J); % SVD-based

Jdmp = J' / (J'*J' + lambda^2*eye(size(J,1))); % damped LS

dq = Jdmp * dx; % lambda ~

0.01-0.1

N = eye(size(J,2)) - pinv(J)*J; % null-space

projector

sv = svd(J); manip = prod(sv); % sqrt(det(J*J'))

5.2 Orientation error

$\Delta\varphi = \text{rotVec}(C_{IE}^* C_{IE}^T)$ (in I) – not difference of Euler angles.

dr = r_des - r_cur;

dphi = rotMatToRotVec(C_des * C_cur'); % I-frame

dx = [dr; dphi]; % 6x1 pose error

5.3 Iterative inverse kinematics (Newton)

function q = inverseKinematics(r_des, C_des, q0, tol)

q = q0; it = 0; max_it = 100; alpha = 0.5; lambda = 1e-3;

J = [jointToPosJac(q); jointToRotJac(q)];

dx = poseErr(q);

while norm(dx) > tol && it < max_it

J = [jointToPosJac(q); jointToRotJac(q)];

q = q + alpha * (J'/(J'*J' + lambda^2*eye(6))) * dx;

dx = poseErr(q); it = it + 1;

% abbRobot.setJointPositions(q); drawnow; % visualise

end

function e = poseErr(q) % nested fn

e = [r_des - jointToPosition(q);

rotMatToRotVec(C_des*jointToRotMat(q'))];

end

end

5.4 Kinematic trajectory control

$\dot{q} = J^+(w^* + K_P \Delta x)$

for t = 0:dt:tEnd

[r_d, v_d] = traj(t); % desired pos & vel

JP = jointToPosJac(q);

dq = pinv(JP) * (v_d + kp*(r_d - jointToPosition(q)));

q = q + dq*dt; % explicit Euler

abbRobot.setJointPositions(q); pause(dt);

end

5.5 Multi-task / null-space

$\dot{q} = J_1^+ w_1 + N_1 (J_2 N_1)^+ (w_2 - J_2 J_1^+ w_1)$

M1 = eye(n) - pinv(J1)*J1;

dq = pinv(J1)*w1 + M1*pinv(J2*M1)*(w2 - J2*pinv(J1)*w1);

% equal priority: stack & weight

dq = pinv([W1*J1; W2*J2]) * [W1*w1; W2*w2];

% secondary objective (e.g. joint centring)

dq = pinv(J)*w + N*(-k0*(q - q_mid));

Recursive n -task: $\dot{q}_i = \dot{q}_{i-1} + (J_i \bar{N}_{i-1})^+ (w_i - J_i \dot{q}_{i-1})$, $N_i = N_{i-1} - (J_i N_{i-1})^+ (J_i N_{i-1})$.

6 Dynamics (fixed base)

$M(q)\ddot{q} + b(q,\dot{q}) + g(q) = \tau + J_C^T F_C$

M = sum(J_Si^T mi JSi + J_Ri^T I @ Si JSi

b = sum(J_Si^T mi JSi q + J_Ri^T (I @ Si JSi q + wi x I @ Si wi)

g = -sum(J_Si^T mi i g; I @ = C Ik k @ C I Ik

syms q [6 1] real; syms dq [6 1] real;

M = sym(zeros(6)); b = sym(zeros(6,1)); g = sym(zeros(6,1));

I_g = [0;0;-9.81];

for k = 1:6

T_Ik = ...; C_Ik = T_Ik(1:3,1:3);

r_IS = T_Ik*[r_r_kSK{k};1]; r_IS = r_IS(1:3); % CoH in I

J_S = jacobian(r_IS, q);

```
J_R = ...; % geometric rot.
    Jacobian
I_Th = C_Th*k_Theta{k}*C_Ik';
w = J_R*dq;
dJ_S = dAdt(J_S, q, dq); dJ_R = dAdt(J_R, q, dq);
M = M + J_S'*m(k)*J_S + J_R'*I_Th*J_R;
b = b + J_S'*m(k)*dJ_S*dq + J_R'*(I_Th*dJ_R*dq + cross(w,
    I_Th*w));
g = g - J_S'*m(k)*I_g;
end
M = simplify(M); % can be SLOW -> simplify per term / skip
function dA = dAdt(A, q, dq) % time derivative
    dA = reshape(jacobian(A(:), q)*dq, size(A));
end
```

6.2 Lagrange (symbolic)

```
T = 1/2*q^T*Mq; U = -sum(m_i I_S^T r_{S_i}, d/dt dC/dq - dC/dq = tau
Tkin = 0.5*dq'*M*dq; U = ...;
M = hessian(Tkin, dq); % M
g = jacobian(U, q)';
b = jacobian(jacobian(Tkin,dq)', q)*dq - jacobian(Tkin, q)';
% b = C(q,dq)*dq (Coriolis + centrifugal)
```

6.3 Code generation & sanity checks

```
matlabFunction(M, 'File','M_fun', 'Vars',{q});
matlabFunction(b, 'File','b_fun', 'Vars',{q,dq});
matlabFunction(g, 'File','g_fun', 'Vars',{q});
Mn = M_fun(q0);
issymmetric(round(Mn,10)), all(eig(Mn) > 0) % SPD?
norm(b_fun(q0, zeros(n,1))) % = 0 at rest
% Ndot - 2C skew-symmetric (energy consistency)
```

6.4 Forward simulation

```
x0 = [q0; zeros(n,1)];
[t, X] = ode45(@(t,x) rhs(t,x), [0 5], x0);
function dx = rhs(t, x)
    n = numel(x)/2; q = x(1:n); dq = x(n+1:end);
    tau = controller(t, q, dq);
    ddq = M_fun(q) \ (tau - b_fun(q,dq) - g_fun(q));
    dx = [dq; ddq];
end
% semi-implicit Euler for fixed-step loops:
dq = dq + dt*ddq; q = q + dt*dq;
plot(t, X(:,1:n)); legend('q_1','q_2'); grid on;
```

Energy check without control/damping: $E = \frac{1}{2}\dot{q}^T M \dot{q} + U$ must stay constant (tighten odeset('RelTol',1e-8)).

7 Model-based Control

7.1 Joint space

PD + gravity comp.: $\tau = K_P(q^* - q) - K_D\dot{q} + g(q)$
 Inverse dynamics: $\tau = M(\ddot{q}^* + K_P(q^* - q) + K_D(\dot{q}^* - \dot{q})) + b + g$
 $\tau = kp*(q_d - q) - kd*dq + g_fun(q);$
 $ddq_ref = ddq_d + kp*(q_d - q) + kd*(dq_d - dq);$
 $\tau = M_fun(q)*ddq_ref + b_fun(q,dq) + g_fun(q);$
*% critical damping per joint: kd = 2*sqrt(kp) (unit mass)*

7.2 Task / operational space

```
Λ = (JM-1JT)-1, μ = ΛJM-1b - ΛJq̇, p = ΛJM-1g
τ = JT(Λẇ* + μ + p), ẇ* = ẇ_d + K_PΔx + K_D(w_d - Jq̇)
Minv = M \ eye(n);
Lambda = pinv(J*Minv*J'); % pinv if singular
mu = Lambda*J*Minv*b - Lambda*dJ*dq;
p = Lambda*J*Minv*g;
dw_ref = dw_d + Kp*[r_d - r; dphi] + Kd*(w_d - J*dq);
tau = J'*(Lambda*dw_ref + mu + p);
% + null-space posture: tau = tau + (eye(n) - J'*Jbar')*tau0
Jbar = Minv*J'*Lambda; % dyn. consistent inverse
```

7.3 Hybrid motion/force control

Selection matrices in contact frame C : Σ_P (motion dirs), $\Sigma_f = I - \Sigma_P$
 $S_p = \text{diag}([1 \ 1 \ 0 \ 1 \ 1 \ 1]); S_f = \text{eye}(6) - S_p; \quad \% z = \text{force dir}$
 $S_M = \text{blkdiag}(C_{IC}, C_{IC}) * S_p * \text{blkdiag}(C_{IC}, C_{IC}); \quad \% \text{ to I frame}$

```
S_F = blkdiag(C_IC,C_IC)*Sf*blkdiag(C_IC,C_IC)';
tau = J*(Lambda*S_M*dw_ref + S_F*F_des + mu + p);
```

8 Floating Base / Legged Robots

```
q = [q_b; q_j]; u = [I v_B; B ω_B; q̇_j]; Mḡ + b + g = STτ + JcTFc, S = [0 I]
nb = 6; nj = 12; S = [zeros(nj,nb), eye(nj)];
% contact constraint (point feet): J_c*u = 0, J_c*du + dJ_c*u = 0
Jc = [J_LF; J_RF; J_LH; J_RH]; % stacked 3xN each
Nc = eye(nb+nj) - pinv(Jc)*Jc; % contact null-space
```

8.1 Contact force elimination

```
F_c = -(J_c M-1 JcT)-1 (J_c M-1 (STτ - b - g) + Jcu)
Minv = M \ eye(size(M));
Lc = pinv(Jc*Minv*Jc');
Fc = -Lc*(Jc*Minv*(S'*tau - b - g) + dJc*u);
du = Minv*(S'*tau + Jc'*Fc - b - g);
```

8.2 Inverse dynamics via projection

```
P = I - Jc+Jc; τ* = (PST)+P(Mḡ* + b + g)
P = eye(nb+nj) - pinv(Jc)*Jc;
tau = pinv(P*S') * P * (M*du_des + b + g);
```

8.3 QP-based (whole-body) control

```
min_x 1/2 x^T Hx + f^T x s.t. equality/inequality, x = [ū; Fc; τ]
% least-squares task ||A x - b||^2 -> H = A'*A, f = -A'*b
H = A'*A + 1e-6*eye(nx); f = -A'*bt;
Aeq = [M, -Jc', -S']; beq = -(b+g); % dynamics
Aeq = [Aeq; Jc, zeros(nc,nc+nj)]; beq = [beq; -dJc*u];
% friction pyramid per foot: |Fxl|/|Fyl| <= mu*Fz, Fz >= 0
Af = [1 0 -mu; -1 0 -mu; 0 1 -mu; 0 -1 -mu; 0 0 -1];
opts = optimoptions('quadprog','Display','off');
x = quadprog(H, f, Af, Ain, bin, Aeq, beq, [], [], [], opts);
```

Hierarchical QP: solve task 1, then optimise task 2 in its null space: $x = x_1^* + N_1 z$. Without Optimization Toolbox: use lsqlin/pinv for equality-only problems.

8.4 Support polygon / CoM

```
r_com = zeros(3,1); J_com = zeros(3,n);
for i = 1:nb, r_com = r_com + m(i)*r_S{i}; J_com = J_com + m(i)*J_S{i}; end
r_com = r_com/sum(m); J_com = J_com/sum(m);
inPoly = inpolygon(r_com(1), r_com(2), feet(1,:), feet(2,:));
```

9 Rotorcraft (Quadrotor)

$T_i = b\omega_i^2, Q_i = d\omega_i^2$ (thrust, drag moment)
 $m I \ddot{r} = C_{IB} \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} + m I g, \quad \Theta B \dot{\omega} + B \omega \times \Theta B \omega = B M$

```
% '+' configuration, rotors 1..4 on +x,y,-x,-y (CHECK
% sign/rotation-direction convention of the exercise!)
A = [ b b b b;
      0 1*b 0 -1*b; % roll moment Mx
      -1*b 0 1*b 0; % pitch moment My
      -d d -d d]; % yaw moment Mz
w2 = A \ [T; Mx; My; Mz]; w2 = max(w2, 0); % saturate
w = sqrt(w2);
% dynamics RHS
F_I = C_IB*[0;0;sum(b*w.^2)] + m*[0;0;-9.81];
ddr = F_I/m;
dw_B = Theta \ (M_B - cross(w_B, Theta*w_B));
```

9.1 Cascaded control (hover linearisation)

```
% position PD -> desired accel -> desired attitude + thrust
a_d = ddr_d + Kd*(dr_d - dr) + Kp*(r_d - r);
T = m*(a_d(3) + 9.81); % small-angle
phi_d = (a_d(1)*sin(psi) - a_d(2)*cos(psi)) / 9.81; % roll
th_d = (a_d(1)*cos(psi) + a_d(2)*sin(psi)) / 9.81; % pitch
% attitude PD -> body moments
M_B = Kp_att*[phi_d-phi; th_d-th; psi_d-psi] - Kd_att*w_B;
```

Fixed-wing basics: $L = \frac{1}{2}\rho V^2 S C_L(\alpha), D = \frac{1}{2}\rho V^2 S C_D(\alpha), C_L \approx C_{L0} + C_{L\alpha}\alpha, C_D \approx C_{D0} + k C_L^2$. Trim: solve $f(x^*, u^*) = 0$ with fsolve.

10 Numerics, Linearisation & Plotting

10.1 Linearisation & LQR

```
f = @(x,u) rhs_xu(x,u); % dx = f(x,u)
A = numjac(@(x) f(x,u0), x0); % df/dx at equilibrium
B = numjac(@(u) f(x0,u), u0); % df/du
% symbolic: A = jacobian(fsym, x); B = jacobian(fsym, u);
K = lqr(A, B, Q, R); % Control Toolbox
u = u0 - K*(x - x0);
eig(A - B*K) % all Re < 0 ?
rank(ctrb(A,B)) == size(A,1) % controllable?
```

10.2 Trajectories

```
% cubic between (q0,0) and (qf,0) in time tf
s = @(t) 3*(t/tf).^2 - 2*(t/tf).^3;
qt = @(t) q0 + (qf-q0)*s(t);
% quintic: 10 s^-3 - 15 s^-4 + 6 s^-5 (zero vel & acc)
% circle in task space
r_d = @(t) c + R*[cos(w*t); sin(w*t); 0];
v_d = @(t) R*w*[-sin(w*t); cos(w*t); 0];
% quaternion slerp between qa, qb
if qa'*qb < 0, qb = -qb; end % shortest path
th = acos(min(1, qa'*qb));
qs = (sin((1-s)*th)*qa + sin(s*th)*qb)/sin(th);
```

10.3 Plotting & animation

```
figure; subplot(2,1,1); plot(t, q'); grid on;
xlabel('t [s]'); ylabel('q [rad]'); legend('q1','q2','q3');
plot3(r(1,:), r(2,:), r(3,:), 'LineWidth',1.5); axis equal
for k = 1:numel(t) % replay trajectory
    abbRobot.setJointPositions(Q(:,k)); drawnow; pause(0.01);
end
```

11 Debugging Checklist

- Frame of each vector? Write it in the name: $I, r_{IB}, B, w_{IB}, C_{IB}$.
- Rotation composition order: $C_{IE} = C_{IO}C_{O1} \dots$ (right-multiply going outward).
- ω in I vs. B : $I\omega = C_{IB} B\omega$; Jacobians in exercise are in I .
- Test on random inputs ($q = \text{randn}(6,1)$), tolerance 1e-6, like evaluate_problems.
- Check Jacobians against numJac; check M symmetric p.d.; check $b(q, 0) = 0$.
- Singularities: $\text{rank}(J), \min(\text{svd}(J)) \rightarrow$ use damped pinv.
- Symbolic slow? avoid global simplify; use matlabFunction(...,'Optimize',true).
- Quaternion drift: re-normalise after every integration step.
- dbstop if error, breakpoints, keyboard, whos, size(x).